Rapport Projet 1 - Sokoban (main)

1) Objectif de ce rapport

Ce document explique en detail : - comment fonctionne l'algorithme BFS dans ce projet ; - comment fonctionnent `successorFunction()` et `isGoal()` ; - ou se trouvent dans le code la modelisation de l'etat, la modelisation du noeud et l'interface de jeu ; - si le code suit les regles de `docs/Project 1 Part 1.pdf`.

Remarque importante : la specification de `docs/SPEC.md` est indiquee comme copie verbatim de l'enonce (`Project 1 Part 1.pdf`) et sert de reference de verification.

---

2) Ou sont les parties demandees dans le code

A. Modeling the state

- Fichier : `sokoban.py`
- Classe : `SokobanPuzzle`
- Elements cles :
  - `ACTIONS` (UP, DOWN, LEFT, RIGHT)
  - `__init__` (creation d'etat)
  - `isGoal` (test de but)
  - `successorFunction` (generation des successeurs)

A. Modeling the node

- Fichier : `node.py`
- Classe : `Node`
- Champs : `state`, `parent`, `action`, `g`
- Methodes : `getPath()`, `getSolution()`

B. BFS

- Fichier : `search.py`
- Fonction : `BFS(s, successorsFn, isGoal)`

C. Game interface

- Fichier : `gui.py`
- Classe : `Game`
- Bouton "Resoudre (BFS)" + simulation visuelle de solution

Point d'entree

- Fichier : `main.py`
- Lancement normal : `python main.py`
- Mode console : `python main.py --console`

---

3) Explication detaillee de BFS

Fichier : `search.py`

3.1 Idee generale

BFS (Breadth-First Search) explore les etats par profondeur croissante : - d'abord tous les etats a distance 0, - puis distance 1, - puis distance 2, etc.

Donc, si toutes les actions ont le meme cout (ici cout = 1), le premier but trouve est optimal (cout minimal).

3.2 Structures de donnees

- `ouvert` : file FIFO (`deque`) = liste Open du cours.
- `ferme` : liste des noeuds deja developpes = Closed.
- `ouvertSet` et `fermeSet` : optimisation pour tester rapidement "deja vu".

3.3 Deroulement de la fonction

1. Creer la racine `Node(s, None, None)`.
2. Si l'etat initial est deja but : retourner `(racine, 0)`.
3. Mettre la racine dans `ouvert`.
4. Tant que `ouvert` n'est pas vide :
   - incrementer `steps` (nombre de noeuds developpes) ;
   - retirer le noeud en tete de file ;
   - l'ajouter a `ferme` ;
   - generer ses successeurs avec `successorsFn` ;
   - pour chaque successeur, si l'etat n'est ni dans Open ni dans Closed :
     - si c'est un but, retourner `(enfant, steps)` ;
     - sinon l'ajouter a Open.
5. Si Open se vide sans but : retourner `(None, steps)`.

3.4 Sens de `steps`

`steps` = nombre d'iterations de la boucle BFS, donc nombre de noeuds developpes. Ce n'est pas la longueur de la solution ; la longueur de solution est `g`.

---

4) Explication detaillee de successorFunction

Fichier : `sokoban.py`

4.1 Role

`successorFunction()` renvoie tous les coups legaux depuis un etat courant sous forme de paires : `(action, successeur)`.

4.2 Symboles manipules

- `R` : joueur
- `0` : mur

- : case vide
- `S` : cible
- `B` : caisse
- `.` : joueur sur cible
- `*` : caisse sur cible

4.3 Logique par direction

Pour chaque action parmi `UP`, `DOWN`, `LEFT`, `RIGHT` : - Calculer `n1` = case devant le joueur. - Si `n1` est libre (ou `S`) : deplacement simple du joueur. - Si `n1` contient une caisse (`B` ou `*`) : - regarder `n2` (derriere la caisse) ; - pousser autorise seulement si `n2` est libre (ou `S`). - Sinon (mur ou impossible) : pas de successeur pour cette action.

4.4 Construction du nouvel etat

La methode `nouvelle_grille(...)` construit une nouvelle grille (pas de mutation de l'etat parent) : - la case quittee par le joueur redevient ou `S` ; - la case d'arrivee du joueur devient `R` ou `.` ; - en cas de poussee, la case d'arrivee de la caisse devient `B` ou `*`.

Cette partie porte les regles de Sokoban : - pas de passage a travers mur, - pas de tirage de caisse, - pas de poussee de deux caisses, - poussee seulement si la case derriere est libre.

---

5) Explication detaillee de isGoal

Fichier : `sokoban.py`

`isGoal()` retourne vrai quand il ne reste plus aucun `B` dans la grille.

Interpretation logique : - `B` = caisse hors cible, - `*` = caisse sur cible.

Donc "plus de `B`" signifie "toutes les caisses sont sur cibles".

---

6) Modelisation de l'etat (A.1)

Fichier : `sokoban.py`

6.1 Representation

- Un etat est une grille stockee en tuple de chaines (`self.grid`) + position joueur (`self.player_pos`).
- Le tuple est immuable et hashable, utile pour sets de BFS.

6.2 Egalite et hash

- `__eq__` compare les grilles.
- `__hash__` hash la grille.

Ceci permet de detecter efficacement les etats deja vus.

6.3 Operations d'etat

- `successorFunction()` genere les transitions valides.
- `isGoal()` teste le but.

---

7) Modelisation du noeud (A.2)

Fichier : `node.py`

Un `Node` contient : - `state` : etat Sokoban, - `parent` : noeud parent, - `action` : action appliquee, - `g` : cout cumule (ici profondeur, car cout de pas = 1).

Methodes : - `getPath()` remonte les parents et renvoie la suite d'etats initial -> but. - `getSolution()` remonte les actions initial -> but.

---

8) Interface de jeu (C)

Fichier : `gui.py`

8.1 Ce qu'elle fait

- Affiche menu + niveaux.
- Permet de jouer manuellement (fleches).
- Lance BFS via bouton/touche.
- Affiche resultat : cout, steps, temps.
- Simule la solution automatiquement.

8.2 Point cle d'architecture

La GUI ne recode pas les regles : - `bouger()` lit `successorFunction()` et applique seulement un successeur valide.

Donc les regles restent centralisees dans `SokobanPuzzle`.

---

9) Audit de conformite a Project 1 Part 1

Reference d'audit : `docs/SPEC.md` (indique comme copie verbatim de l'enonce PDF).

9.1 A.1 Modeling state

- Exige : symboles R, O, espace, S, B, ., * -> OK (`sokoban.py`, `levels.py`).
- Exige : mouvements UP/DOWN/LEFT/RIGHT -> OK (`ACTIONS`).
- Exige : `init`, `isGoal`, `successorFunction` -> OK (`__init__`, `isGoal`, `successorFunction`).

9.2 A.2 Modeling node

- Exige : `state`, `parent`, `action`, `g` -> OK (`node.py`).
- Exige : `getPath`, `getSolution` -> OK (`node.py`).

9.3 B BFS

- Exige : BFS selon pseudo-code -> OK (`search.py` avec Open FIFO + Closed + test de but).
- Optimisation sets presente mais sans changer la logique BFS -> conforme au contrat technique.

9.4 C Game interface

- Exige : simulation solution + cout + nombre de steps -> OK (`gui.py`, `main.py`).

9.5 Conclusion audit

Le code suit les regles principales de l'enonce et de `docs/SPEC.md` pour A, B, C.

---

10) Commandes utiles

- Lancer le jeu :
  - `python main.py`
- Lancer en mode console :
  - `python main.py --console`
- Lancer les tests :
  - `python test_core.py`

---

11) Notes pour transformer ce rapport en PDF

Sur cette machine, `pandoc` n'est pas installe. Options simples : 1. Ouvrir ce fichier Markdown dans VS Code et utiliser "Print" vers PDF. 2. Installer pandoc puis exporter en PDF.

Commande pandoc type (si installe plus tard) : `pandoc docs/RAPPORT_PROJECT1_MAIN.md -o docs/RAPPORT_PROJECT1_MAIN.pdf`